项目创新说明

项目名称：多智能体低开销通信与共享记忆实验系统

参赛队伍：斑马小队

我们面向连续问答与代码执行，围绕外部信息增量接入、题内状态管理和事实与经验双层记忆形成三项技术贡献。下文分别说明最接近的已有工作、已有能力、新增机制及验证依据。代码与记录路径均相对于 examples/competition_agents/。

一 面向外部工具交互的隐状态增量协作机制

最接近的已有工作 LatentMAS 的内部状态推理与角色间工作记忆传递机制。

已有方案具备的能力 原论文已有题内工作记忆和增量计算：角色生成连续内部表示，后继角色接续各层缓存，继续处理新输入和内部推理状态。我们接入的公开基准主流程围绕给定题目完成角色推理，未提供本项目中外部检索、持久记忆和执行反馈的接入与登记机制。

本项目新增的技术贡献 我们让角色在内部状态中完成规划和推理，同时通过必要参数连接外部工具。检索函数和执行器不能直接读取模型状态，因此检索角色依据继承的状态输出动作与查询参数，代码执行输出待执行程序；完整规划和推理过程继续保留在计算状态中。调用返回的知识、记忆及实际执行反馈以字段形式补入已有状态，角色随后继续计算。

同题事实以统一编号和内容指纹登记。已经处理且未变化的事实沿用原状态，新增或内容改变的事实追加编码；只有前向计算完成后才登记已处理。当前选中的证据另行记录，避免把“曾经编码过”误当作“本轮正在采用”。这项机制把外部信息持续到来的处理过程与角色状态接续连接起来，减少完整中间结果的文本转换和重复输入。

代码位置

代码位置	对应实现
competition_agents/llm/continuous_state_chat.py	TOOL_INTERFACE、_call_role：内部状态工作、工具参数输出及反馈后的接续。
competition_agents/llm/resident_chat.py	_evidence：事实编号与内容指纹、增量编码登记和当前证据选择。
competition_agents/unified_collaboration.py	_lookup、_execute：知识与记忆检索、外部执行和结果反馈。
tests/test_latentmas_upstream.py	检查真实内部计算、各层缓存接续，以及工具、记忆和执行器的完整流程。

验证依据 tests/test_latentmas_upstream.py 提供工具与记忆接入的流程核查；outputs/report_full_20261002/server-results/raw/ 中，各模型 qa/ 与 codeact/ 的 latent/result.json 保存实际输入、工具参数、反馈内容和缓存接续记录，可核对外部结果是否进入模型及已处理事实是否复用。

已有工作来源：https://arxiv.org/abs/2511.20639；https://github.com/Gen-Verse/LatentMAS。接入提交 9a9e4d3，来源及兼容修改记录见 competition_agents/vendor/latentmas/provenance.json。

二 角色消费驱动的题内状态生命周期管理

最接近的已有工作 LatentMAS 的隐状态多智能体协作实现。

已有方案具备的能力 该实现通过连续表示进行中间计算，接续各层键值缓存，让后继角色继续使用已有计算状态，并提供保留部分后缀的截断选项。我们在此基础上增加按内容类型和消费条件管理状态的机制。

本项目新增的技术贡献 我们把工作状态与内容类型、所属角色、消费时点和保留期限关联，区分原始证据、角色专用指令、工具控制信息及内部交付位置。题内回收原型在登记的消费条件满足后移除到期位置，保留问题、事实正文及用于后续交付的末端状态；模型仍可沿用保留的计算结果。

回收后，物理缓存长度可以缩短，逻辑位置继续增长。我们根据保留位置构造注意力掩码，避免把删除后的存储长度当作原始位置；混合结构中的递归状态保持原张量。键和值逐个替换，回收时的临时复制也计入峰值。同时，我们释放已不再被使用的输出引用，在事实与历史写入记忆后回收整题工作状态。

代码位置

代码位置	对应实现
experiments/role_private_cache_probe.py	RolePrivateCache 与 PrivateInstructionManager：位置登记、消费条件、题内回收和掩码维护。
competition_agents/latent_hidden/state_lifecycle.py	TaskStateLifecycle：公共前缀保存与题末状态恢复。
competition_agents/llm/latentmas_chat.py	StateForward、finish_task：临时输出释放及任务收尾。
tests/test_role_private_cache_probe.py tests/test_output_buffer_lifecycle.py	检查证据位置保护、递归状态保留、裁剪后的接续，以及输出缓冲释放前后的一致性。

验证依据 题内回收的三模型前三题记录位于 outputs/server_role_private_v3_20261003/server-results/role-private-three-v3-20261003/，各模型子目录保存 summary.json、tasks.json 和 live_boundaries.json，可逐题核对阶段缓存、回收位置、峰值及最终答案。

默认主流程已接入临时输出释放和题末回收；题内细粒度回收在独立原型中验证。早期潜在位置的回收属于近似策略，我们通过保留位置检查与最终答题结果共同检验其影响。

已有工作来源：https://github.com/Gen-Verse/LatentMAS；接入提交 9a9e4d3，来源及兼容修改记录见 competition_agents/vendor/latentmas/provenance.json。

三 完整关系驱动的事实与经验双层超图复用

最接近的已有工作 HyperGraphRAG 的多元关系组织与关联检索。

已有方案具备的能力 HyperGraphRAG 将多个实体的关系组织为超边，并结合关联证据回答问题。我们面向连续任务，将完整关系组织与跨题事实写回、经验记录和再次检索连接起来。

本项目新增的技术贡献 我们为事实保留统一身份与来源作用域，把主体、关系、客体、极性、限定条件和出处一起组织。别名与指代在来源范围内绑定，关联召回后完整交付关系；重复标题集中存入来源表，避免接收角色再次拼接主体与所属对象。

情景事实层写回实际取得的关系，包括本题暂未采用、后续可能需要的事实；经验层保存任务中观察到的检索路径、适用条件和采用反馈，供后续角色选择处理路径。角色先检查历史与已存事实，信息不足时再补查知识库，知识库和记忆使用同一套关系表示。跨题信息由持久记忆承接，每题结束后清理整题工作缓存。

代码位置

代码位置	对应实现
hypergraph_memory_method/prepared_knowledge.py hypergraph_memory_method/source_context.py	来源作用域建图、实体身份绑定、完整关系与独立来源表组装。
competition_agents/evidence_memory.py	SessionEvidenceMemory：已见事实写回、实体关联召回和历史承接。
hypergraph_memory_method/retrieval_experience.py	RetrievalExperienceMemory：检索轨迹、适用条件、采用反馈与经验存储。
competition_agents/collaboration_memory.py competition_agents/unified_collaboration.py	记忆优先补查、已观察事实写回与跨题提交。

验证依据 tests/test_memory_source_alignment.py、tests/test_source_context_packet.py 和 tests/test_retrieval_experience.py 检查后续关系取回、别名与限定交付、经验作用域及重复写入。

三个模型的十轮问答及十项代码任务结果保存在 outputs/report_full_20261002/server-results/raw/。各模型目录中的 qa/ 和 codeact/ 按三种协作方式分别保存 result.json，可核对实际输入、交付、记忆和工具轨迹。经验层当前采用检索路径提示，证据是否足够由检索或推理角色结合当前任务判断。

已有工作来源：HyperGraphRAG，https://arxiv.org/abs/2503.21322。